```python
from queue import PriorityQueue

def best_first_search(graph, start, target, heuristics):
    visited = set()
    pq = PriorityQueue()

    # (heuristic_value, current_node, path)
    pq.put((heuristics[start], start, [start]))
    visited.add(start)

    while not pq.empty():
        h, node, path = pq.get()

        # Goal Test
        if node == target:
            return path, h

        # Visit neighbors
        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                pq.put((heuristics[neighbor], neighbor, path + [neighbor]))

    return None, None


# Example Usage
if __name__ == "__main__":

    graph = {
        'A': ['B', 'C'],
        'B': ['D', 'E'],
        'C': ['F'],
        'D': [],
        'E': [],
        'F': []
    }

    heuristics = {
        'A': 10,
        'B': 8,
        'C': 5,
        'D': 6.
```

```
        'F': []
    }

    heuristics = {
        'A': 10,
        'B': 8,
        'C': 5,
        'D': 6,
        'E': 4,
        'F': 0
    }

    path, cost = best_first_search(graph, 'A', 'F', heuristics)

    if path:
        print(f"Path found by Best-First Search: {path}")
        print(f"Target Heuristic: {cost}")
    else:
        print("No path found.")
print("Name: Moulya M P")
print("Register Number: 24BECS167")
```

Run

Output

```
Path found by Best-First Search: ['A', 'C', 'F']
Target Heuristic: 0
Name: Moulya M P
Register Number: 24BECS167

=== Code Execution Successful ===
```